

Sınıfı Kapatmak: Güvenlik Bulgularının Tip Tasarımındaki Karşılığı

Ömer Faruk Yavuz

August 2026

Özet

OWASP Top 10, en tehlikeli on açığın listesi değil, ölçülebilirlik ile saha sezgisi arasında bilinçli olarak kurulmuş bir uzlaşmadır: kategorilerin sekizi katkı verisinden, ikisi topluluk anketinden seçilir. Bu yapı listenin kendi belgesinde açıkça kabul edilir ve doğrudan bir sonucu vardır: Top 10 tek başına doğrulanabilir bir denetim listesi olarak kullanılamaz. Listenin sıralaması, bir zafiyet türünün kaç kez bulunduğu değil, uygulamaların yüzde kaçında en az bir kez bulunduğu dayanır; bu tercih, otomatik araçların ürettiği yüksek bulgu hacminin insan testçi verisini bastırmasını engeller ve saldırganın bir kategoriye sömürmek için tek bir örneğe ihtiyaç duyması olgusuyla gerekçelendirilir. 2021 sürümünde kategori adları semptomdan kök nedene kaydırılmış, analiz edilen zayıflık sayısı yaklaşık otuzdan dört yüze çıkarılmıştır. OWASP’ın asıl işlevsel katkısı tek bir liste değil, yazılım yaşam döngüsünün her aşamasına karşılık gelen bir belge ve araç kümesidir; gereksinim yazımında ASVS, günlük uygulamada Cheat Sheet Series, arayüz katmanında API Security Top 10 bu kümenin taşıyıcı parçalarıdır. Güvenlik alanının ürettiği bulguların önemli bir bölümü, tip tasarımı ve arayüz tasarımı disiplinlerinin bağımsız olarak vardığı sonuçlarla örtüşür: nesne düzeyinde yetkilendirme eksikliği, girdi tipiyle alan tipinin karıştırılması ve kanonikleştirilmemiş tanımlayıcı üretimi bunun üç örneğidir. Bu örtüşme, güvenlik kontrollerinin sonradan eklenen denetimler yerine imza ve tip düzeyinde zorunlu kılınmasının neden daha dayanıklı olduğunu açıklar.

Giriş

Uygulama güvenliği alanında en yaygın bilinen belge OWASP Top 10'dur. OWASP (*Open Worldwide Application Security Project*), uygulama güvenliği alanında açık kaynaklı belge, standart ve araç üreten kâr amacı gütmeyen bir vakıftır. Top 10, web uygulamalarına yönelik en kritik güvenlik risklerini on kategori hâlinde derleyen bir farkındalık belgesidir ve yaklaşık dört yılda bir güncellenir.

Belgenin yaygınlığı, sistematik bir yanlış kullanıma da yol açmıştır: Top 10 sıklıkla bir uyumluluk kontrol listesi gibi ele alınır, şartnamelere “OWASP Top 10 uyumludur” maddesi olarak yazılır ve denetimlerde kabul kriteri sayılır. Bu kullanım, belgenin kendi metodoloji bölümünde açıkladığı üretim biçimiyle çelişir. Listenin nasıl üretildiğini bilmek, dolayısıyla listenin içeriğini bilmekten daha belirleyicidir.

Metin boyunca üç kısaltma tekrar edecektir. CWE (*Common Weakness Enumeration*), yazılım zayıflıklarının tür düzeyinde sınıflandırıldığı bir katalogdur; “SQL enjeksiyonu” ya da “yol geçişi” gibi bir hata sınıfını adlandırır. CVE (*Common Vulnerabilities and Exposures*), belirli bir ürünlerdeki belirli bir zafiyetin tekil kayıdır; CWE tür, CVE ise o türün somut bir örneğidir. CVSS (*Common Vulnerability Scoring System*), bir CVE'nin sömürülebilirliğini ve teknik etkisini sayısal olarak puanlayan ölçektir. NVD (*National Vulnerability Database*), ABD Ulusal Standartlar ve Teknoloji Enstitüsü tarafından işletilen ve CVE kayıtlarını CVSS puanlarıyla birlikte tutan veritabanıdır.

Bu not iki şeyi kapsar: Top 10'un üretim yöntemi ve bu yöntemin listenin kullanım sınırları üzerindeki sonuçları; ve OWASP ekosisteminin yazılım yaşam döngüsündeki karşılıkları. Ayrıca sunucu tarafı web servislerinde bu bulguların somut karşılıkları ele alınır. Notun kapsamı dışında kalanlar: ağ ve altyapı güvenliği, kriptografik ilkel tasarımı ve kurumsal güvenlik yönetişi.

Top 10'un Üretim Yöntemi

Sekiz kategori veriden, iki kategori anketten

Kategorilerin sekizi katkı verisinin analizinden, ikisi topluluk anketinden seçilir. Gerekçe, verinin doğası gereği geriye dönük olmasıdır: yeni bir zafiyet türünün keşfedilmesi, bunun için test yöntemi geliştirilmesi, yöntemin araçlara gömülmesi ve araçların geniş bir uygulama kümesinde çalıştırılması yıllar alır. Bir zayıflık ölçeklenebilir biçimde test edilebilir hâle geldiğinde, artık yeni değildir. Saf veriye dayalı bir liste bu nedenle son yılların eğilimlerini yapısal olarak kaçırır. Anket, sahadaki uzmanların henüz veriye yansımamış gözlemlerini listeye taşımak için kullanılır (OWASP, 2021).

Bunun somut örneği 2021 sürümünde sunucu tarafı istek sahteciliğidir. Bu kategori veride düşük bir insidans oranına sahip olmasına rağmen, topluluk anketinde birinci sırada oy aldığı için listeye alınmıştır; belge bunu, “veri henüz göstermiyor ama uzmanlar önemli diyor” durumu olarak açıkça niteler (OWASP, 2021).

Kategori yapısı: semptomdan kök nedene

2021 sürümünde üç yeni kategori açılmış, dört kategoride ad ve kapsam değişikliği yapılmış, bazı kategoriler birleştirilmiştir. Değişikliklerin ortak ilkesi, adların semptom yerine kök nedene işaret etmesidir. “Sensitive Data Exposure” adının “Cryptographic Failures” olarak değiştirilmesi bu ilkenin örneğidir: hassas verinin ifşası bir sonuçtur, nedeni ise çoğunlukla kriptografik bir kusurdur. Kök nedene odaklanmak, giderme rehberliğinin daha kesin yazılabilmesini sağlar; semptom düzeyindeki bir ad, aynı sonuca yol açan birbirinden bağımsız hataları tek başlık altına toplar (OWASP, 2021).

Aynı sürümde XML dış varlık saldırıları (XXE) ayrı bir kategori olmaktan çıkarılıp güvenlik yapılandırma hatalarının içine alınmıştır; sınıflandırma gerekçesi, XXE’nin özünde bir ayrıştırıcı yapılandırma sorunu olmasıdır. Siteler arası betik çalıştırma (XSS) de ayrı kategori olmaktan çıkıp enjeksiyon kategorisinin parçası hâline gelmiştir.

Kırık erişim kontrolü beşinci sıradan birinci sıraya yükselmiştir. Bu kategoriye eşlenen 34 CWE, uygulamalarda diğer bütün kategorilerden daha fazla örnekle görülmüştür (OWASP, 2021).

Frekans yerine insidans oranı

Metodolojinin en öğretici kısmı, ölçünün ne olduğudur. Üç veri kaynağı ayırt edilir: ham araç çıktısı, insan destekli araç kullanımı (*Human-assisted Tooling*) ve araç destekli insan testi (*Tool-assisted Human*).

Araçlar bir zafiyet türünün her örneğini yorulmadan aramaya devam eder. Sistemik bir XSS sorunu bulunan tek bir uygulamada binlerce bulgu üretilebilir. İnsan testçi ise aynı durumda üç dört örnek bulup durur; sorunun sistemik

olduğunu tespit edip uygulama genelinde giderme önerisi yazmak için her örneği bulması gerekmez, zaten zamanı da yoktur.

Bu iki veri kümesi bulgu sayısı üzerinden birleştirilirse araç verisi insan verisini boğar. Belge, XSS'in yıllarca listelerde çok yüksekte çıkmasını doğrudan buna bağlar: etkisi genellikle düşük ile orta arasındayken sıralaması yüksektir, çünkü ölçülen şey riskin kendisi değil bulgu hacmidir. Test edilmesi kolay olan zafiyetlerin daha çok test edilmesi de aynı yönde çalışır (OWASP, 2021).

Kullanılan ölçü bunun yerine insidans oranıdır: bir zafiyet türünün en az bir örneğinin, test edilen uygulama kümesinin yüzde kaçında bulunduğu. Bir uygulamada dört örnek mi yoksa dört bin örnek mi bulunduğu hesaba katılmaz. Bu tercihin gerekçesi risk temellidir: saldırganın bir kategoriye başarıyla sömürmek için tek bir örneğe ihtiyacı vardır (OWASP, 2021).

Sömürülebilirlik ve etki puanlaması

Insidans oranı olasılık tarafını verir; risk sıralaması için etki tarafı da gerekir. Sömürülebilirlik ve teknik etki puanları CVSS'ten türetilir. Veri, OWASP Dependency-Check aracıyla NVD'den çıkarılmıştır: bir CWE'ye eşlenmiş 125 bin CVE kaydı ve bir CVE'ye eşlenmiş 241 benzersiz CWE bulunmaktadır; bu eşlemelerin yaklaşık yarısında CVSSv3 puanı vardır, kalanında yalnızca CVSSv2 mevcuttur. İki sürümün formülleri ve ölçek aralıkları farklı olduğu için, her CWE'nin ortalama puanı iki popülasyonun oranına göre ağırlıklandırılarak hesaplanmıştır (OWASP, 2021).

Bu yöntemin bir sınırı, belgenin kendisinde belirtilir. Savunmasız ve güncelliğini yitirmiş bileşenler kategorisine eşlenen CWE'lere hiçbir CVE eşlenmediği için, bu kategoriye varsayılan olarak 5,0 sömürü ve etki ağırlığı atanmıştır (OWASP, 2021). Yani sıralamadaki bir kategorinin puanı, ölçülmüş bir değer değil, veri yokluğunda seçilmiş bir sabittir.

Veri kümesinin kapsamı

Önceki sürümlerde veri toplama, önceden belirlenmiş yaklaşık 30 CWE'lik bir alt kümeye odaklanıyor, kuruluşlara ek bulguları bildirme alanı bırakılıyordu. Uygulamada kuruluşlar neredeyse yalnızca o 30 maddeye bakıyor, gördükleri başka zayıflıkları nadiren ekliyordu; yani ölçüm aracının kendisi ölçülen şeyi belirliyordu. 2021 sürümünde kısıt kaldırılmış ve analiz edilen CWE sayısı yaklaşık 400'e çıkmıştır. Kategori başına ortalama 19,6 CWE düşer; en az bir CWE ile sunucu tarafı istek sahteciliği, en çok 40 CWE ile güvensiz tasarım kategorisi yer alır (OWASP, 2021).

Veri; test hizmeti veren firmalardan, ödül avı (*bug bounty*) platformlarından ve kendi iç test verisini paylaşan kuruluşlardan gelir ve 500 binden fazla uygulamayı kapsar. Katkı veren kuruluşlar arasında GitLab, HackerOne, Veracode, Cobalt.io ve Contrast Security bulunur (OWASP, 2021).

Bu kapsamın kendi yanlılığı vardır ve belge bunu kabul eder: sonuçlar, büyük ölçüde otomatik olarak test edilebilen şeylerle sınırlıdır. Test edilemeyen bir zayıflık, yaygın olsa bile veride görünmez.

2025 sürümü

Sekizinci sürüm olan OWASP Top 10:2025, Kasım 2025'te duyurulmuş ve nihai hâli Ocak 2026'da yayımlanmıştır. Analiz edilen CWE sayısı yaklaşık 400'den 589'a çıkmış, iki yeni kategori açılmış ve sunucu tarafı istek sahteciliği ayrı kategori olmaktan çıkarılıp kırık erişim kontrolü içine alınmıştır (OWASP, 2025). Metodolojinin temeli — sekiz kategorinin veriden, ikisinin anketten seçilmesi ve insidans oranının ölçü olarak kullanılması — korunmuştur; dolayısıyla bu notun metodolojiye ilişkin tespitleri sürümden bağımsızdır.

Belgenin Türkçe çevirisi 2021 sürümüne kadar mevcuttur. Türkçe okunacaksa metodoloji bölümü için 2021 sürümü kullanılabilir, ancak sıralamanın güncel olmadığı bilinerek okunmalıdır.

Listenin Kullanım Sınırı

Üretim yönteminden çıkan sonuç doğrudandır: Top 10 bir kontrol listesi değildir. Bunun üç ayrı nedeni vardır.

Birincisi, kategorilerin ikisi ölçümle değil oyla seçilmiştir; belge bunu bir kusur olarak değil, bilinçli bir tasarım kararı olarak sunar. İkincisi, kategoriler kök nedene göre gruplandığı için her biri onlarca farklı zayıflığı kapsar; “A01 uyumludur” ifadesi, kapsanan 34 CWE’nin hangilerinin ele alındığını söylemez. Üçüncüsü, liste farkındalık belgesi olarak tasarlanmıştır; maddeleri sınanabilir ifadeler değil, risk başlıklarıdır.

Doğrulanabilir madde listesi gereken yerlerde — şartname yazımı, kabul kriteri tanımı, tedarikçi denetimi — ASVS (*Application Security Verification Standard*) kullanılır. ASVS, modern web uygulamaları ve servisleri tasarlanırken, geliştirilirken ve test edilirken karşılanması gereken güvenlik kontrollerini üç olgunluk seviyesinde ve numaralı maddeler hâlinde tanımlar (OWASP ASVS). Bir maddenin karşılanıp karşılanmadığı sınanabilir; Top 10 kategorisinin karşılanıp karşılanmadığı sınanamaz.

Bu ayrım, iki belgenin rekabet hâlinde olduğu anlamına gelmez. Top 10 nereye bakılacağını, ASVS neyin doğrulanacağını söyler.

OWASP Proje Ekosistemi

OWASP çatısı altında yüzlerce proje bulunur ve bunlar dört seviyeli bir olgunluk merdiveninde konumlandırılır: Incubator (deneysel), Lab (kullanılabilir), Production (üretim düzeyinde), Flagship (vakıf için stratejik olarak taşıyıcı). Bu seviye bilgisi, bir projeye ne kadar güvenileceğini belirlemede kullanılır.

Aşağıdaki gruplandırma yazılım yaşam döngüsündeki konuma göredir.

Gereksinim ve tasarım aşaması

- **ASVS** — güvenlik gereksinimlerinin numaralı maddeler hâlinde tanımlandığı çerçeve. Şartname ve kabul kriteri buradan yazılır.
- **Proactive Controls** — Top 10'un pozitif karşılığı. Top 10 nelerin yanlış gittiğini, Proactive Controls hangi on uygulamanın bu kategorilerin çoğunu kapattığını anlatır. Geliştirici için daha doğrudan kullanışlıdır ve Top 10'a kıyasla belirgin biçimde az bilinir.
- **Threat Dragon** — tehdit modelleme aracı. Tasarım aşamasında “burada ne ters gidebilir” sorusunu veri akış şemasına bağlar.

Güvensiz tasarımın 2021'de ayrı bir kategori olarak açılmasının gerekçesi tam olarak bu aşamayı işaret eder: güvensiz bir tasarım, ne kadar kusursuz kodlanırsa kodlansın giderilemez, çünkü tanım gereği gereken güvenlik kontrolü hiç oluşturulmamıştır (OWASP, 2021).

Uygulama aşaması

Cheat Sheet Series, belirli konularda yoğunlaştırılmış uygulama rehberleridir. ASVS ve Proactive Controls ile arasında kurumsallaşmış bir çalışma kanalı vardır: bu belgelerdeki bir madde için karşılık gelen rehber sayfası eksikse Cheat Sheet ekibi onu yazar, sayfa hazır olduğunda ASVS geri referans verir. Üç proje birbirine kilitlidir — ASVS neyin gerektiğini, Cheat Sheet nasıl yapılacağını, Proactive Controls neye öncelik verileceğini söyler.

Test aşaması

- **WSTG** (*Web Security Testing Guide*) — web uygulamalarının ve servislerin güvenlik testi için kapsamlı metodoloji. Sızma testi süreçlerinin fiilî referansıdır.
- **Dependency-Check** — proje bağımlılıklarını bilinen CVE kayıtlarıyla eşleştiren araç. Top 10 2021'in puanlama tarafındaki NVD çıkarımı da bu araçla yapılmıştır.
- **DefectDojo** — zafiyet yönetimi; farklı araçların çıktısını tek yerde toplar ve yinelenen bulguları birleştirir.

- **Amass** — saldırı yüzeyi keşfi ve alt alan adı envanteri.

ZAP ve Security Knowledge Framework, sıklıkla OWASP projesi olarak listelenmelerine rağmen resmî olarak OWASP çatısından ayrılmıştır. ZAP hâlâ en yaygın açık kaynaklı dinamik test aracıdır; değişen yalnızca kurumsal bağlantısıdır.

Alan-özel listeler

- **API Security Top 10** — arayüz katmanı yazan biri için ana listeden daha alakalıdır. Nesne düzeyinde yetkilendirme atlatma, aşırı veri ifşası ve toplu atama gibi yalnızca API mimarilerinde ortaya çıkan sınıfları kapsar.
- **MAS** (*Mobile Application Security*) — mobil uygulamalar için standart (MASVS) ve test rehberinden (MASTG) oluşan belge serisi.
- **LLM Top 10** ve **AI Security and Privacy Guide** — büyük dil modeli tabanlı uygulamalar için; klasik makine öğrenmesi modelleri ayrı bir listede ele alınır.
- **Automated Threat Handbook** — bot trafiği, veri kazıma ve kimlik bilgisi doldurma gibi otomatik tehditlerin taksonomisi. Klasik Top 10 bu sınıfı kapsamaz.

Bunlara ek olarak CI/CD hatları, Kubernetes, Docker, sunucusuz mimariler, istemci tarafı, veri katmanı ve akıllı sözleşmeler için ayrı “Top 10” listeleri bulunur. Bu listelerin pratik faydası, ilgili mimariyle fiilen çalışılıyor olmasına bağlıdır.

Tedarik zinciri

CycloneDX, tedarik zinciri riskinin azaltılması için tam yığın malzeme listesi (*Bill of Materials*) standardıdır; yazılım malzeme listesi (SBOM) biçimi olarak SPDX’in başlıca alternatifidir ve kamu ihalelerinde giderek daha sık talep edilmektedir. **Dependency-Track**, üretilen CycloneDX çıktılarını sürekli izleyerek yeni yayımlanan CVE kayıtlarını mevcut envanterle eşleştirir. **SCVS** (*Software Component Verification Standard*) ise ASVS’in tedarik zinciri karşılığıdır.

Bu grubun ağırlığı artmaktadır: 2025 sürümünde bileşen kaynaklı riskler ayrı bir tedarik zinciri kategorisine dönüştürülmüştür (OWASP, 2025).

Süreç ve kültür

- **SAMM** (*Software Assurance Maturity Model*) — bir kuruluşun yazılım güvenliği duruşunu ölçmek ve aşamalı olarak iyileştirmek için olgunluk modeli.
- **Security Champions Guide** — her geliştirme ekibinde bir güvenlik temsilcisi yetiştirme modeli. Ayrı bir güvenlik ekibi kuramayacak ölçekteki yapılar için uygulanabilir olan yaklaşım budur.

- **DevSecOps Guideline** — güvenlik kontrollerinin sürekli tümleştirme ve dağıtım hattına yerleştirilmesi.

Öğrenme ortamları

Juice Shop, kasıtlı olarak zafiyetli, Node.js ile yazılmış modern bir web uygulamasıdır; içerdiği zafiyetler meydan okumalar hâlinde düzenlenmiş ve Top 10, ASVS, API Top 10, Automated Threat Handbook ile MITRE CWE kataloğuna göre sınıflandırılmıştır. Bu sınıflandırma, aracı bir oyundan öğretim materyaline dönüştüren şeydir. WebGoat, PyGoat ve WrongSecrets aynı fikrin farklı dillerdeki karşılıklarıdır; sonuncusu özellikle sızdırılmış sır (*secret*) senaryolarına odaklanır.

Developer Guide, bütün bu projelerin hangi aşamada nereye oturduğunu anlatan üst belgedir; ekosisteme giriş noktası olarak kullanılır.

Sunucu Tarafı Web Servislerinde Karşılıklar

Bu bölümde, yukarıdaki kategorilerin yaygın bir sunucu tarafı yığınındaki — Go ile yazılmış HTTP servisleri, ilişkisel veritabanı, S3 uyumlu nesne depolama, WebSocket ve tek sayfa uygulaması — somut karşılıkları ele alınır.

Nesne düzeyinde yetkilendirme

BOLA (*Broken Object Level Authorization*), bir isteğin kimlik doğrulamasından geçmiş olmasına rağmen, eriştiği kaydın o kullanıcıya ait olup olmadığının denetlenmemesidir. API Security Top 10'un birinci maddesidir ve sunucu tarafındaki en yaygın açık sınıftır.

Tipik hâli şudur:

```
func GetAsset(c *gin.Context) {
    id := c.Param("id")
    asset, err := repo.FindByID(id)    // kim istiyor, kimin varlığı?
    c.JSON(200, asset)
}
```

Kod doğru görünür, kod incelemesinden geçer, testler yeşildir. Eksik olan tek şey, isteği yapanın o kaydı görme hakkının bulunup bulunmadığıdır. Kimlik doğrulama (*authentication*) yapılmıştır — kullanıcı giriş yapmıştır; yetkilendirme (*authorization*) yapılmamıştır — bu kaydın ona ait olup olmadığı sorulmamıştır. Çok kiracılı (*multi-tenant*) bir sistemde bunun karşılığı, bir müşterinin tanımlayıcı tahmin ederek başka bir müşterinin verisini okuyabilmesidir.

Bu sınıfın örnek örnek yamanması, örneklerin unutulacağı ölçüde güvenilmezdir. Sınıfı kapatan çözüm, yetkilendirmeyi çağrı yerine değil imzaya taşımaktır: `FindByID(id)` yerine `FindByID(ctx, tenantID, id)`. Parametre zorunlu olduğu için, kiracı bağlamını geçmeyi unutan kod derlenmez. Denetimin doğru yapılması yerine, yanlış yapılmasının imkânsızlaştırılması hedeflenir.

Toplu atama

Toplu atama (*mass assignment*), istemciden gelen gövdenin doğrudan alan nesnesine bağlanmasıyla ortaya çıkar:

```
var user User
c.ShouldBindJSON(&user)    // {"name":"x","role":"admin"}
```

İstemci, arayüzün göndermesi beklenmeyen alanları da gönderebilir; bağlama işlemi bunları ayırt etmez. Çözüm, her uç nokta için ayrı bir girdi tipi tanımlamak ve alan nesnesine dönüşümü açıkça yazmaktır.

Bu, güvenlik dışı bir gerekçeyle de savunulan bir ayrımdır: girdi tipi ile alan tipi aynı şey değildir, çünkü biri dış arayüzün sözleşmesi, diğeri iç modelin temsilidir. Güvenlik alanı bunu bir zafiyet sınıfı olarak keşfetmiş, tip tasarımı

alanı aynı sonuca bağımsız bir yoldan varmıştır. Bu tür örtüşmeler, güvenlik önerilerinin çoğunun ayrıca maliyet getiren ek işler olmadığını gösterir.

Nesne depolama ve ön imzalı URL’ler

Ön imzalı URL (*presigned URL*), nesne depolamadaki bir kaynağa, imza içinde taşınan yetkiyle süreli erişim veren bağlantıdır. Üç noktada yanlış kullanılır.

Birincisi, ön imzalı URL bir taşıyıcı kimlik bilgisidir (*bearer credential*). Kayıt dosyasına düştüğünde, **Referer** başlığıyla üçüncü tarafa sızdığında ya da hata mesajında görüldüğünde, erişimin kendisi sızmış olur. Bağlantı gizli veri gibi ele alınmalıdır.

İkincisi süredir. Rahat süreler — bir saat gibi — alışkanlık hâline gelir; oysa bir tarayıcının dosyayı indirmesi için gereken süre çoğunlukla dakikalarla ölçülür. Süre, işlemin gerçek ihtiyacına göre belirlenir.

Üçüncüsü nesne anahtarının üretilmesidir. Anahtar kullanıcıdan gelen dosya adından türetiliyorsa, bu doğrudan kanonikleştirme (*canonicalization*) sorunudur: `../` dizileri, Unicode normalizasyon farkları, sondaki nokta ve boşluk karakterleri kapsam dışına yazma ya da beklenmedik çakışma üretebilir. Dosya adını hiç kullanmayıp anahtarı sunucuda üretilen bir UUID’den oluşturmak ve özgün adı ayrı bir sütunda saklamak bu sınıfı bütünüyle kapatır. Yine aynı ilke: doğrulama eklemek yerine, doğrulanacak girdiyi ortadan kaldırmak.

WebSocket

Uzun ömürlü bağlantıda yetkinin tazelenmesi

WebSocket bağlantılarında yetkilendirme çoğunlukla yalnızca el sıkışma (*handshake*) anında yapılır. Bağlantı kurulduktan sonra gelen her mesaj, ilk andaki yetkiyle işlenir. Kullanıcının rolü değiştiğinde, oturumu iptal edildiğinde ya da bir kanala erişimi kaldırıldığında, açık bağlantı bunu görmez ve eski yetkiyle çalışmaya devam eder. Saatlerce hatta günlerce açık kalabilen bağlantılarda yetki, mesaj başına ya da düzenli aralıklarla yeniden değerlendirilmelidir.

Abonelik modeli kullanılıyorsa ve kanal adı istemciden geliyorsa, bu ayrıca bir nesne düzeyinde yetkilendirme yüzeyidir: kullanıcının başka bir kiracının kanalına abone olup olamadığı sınanmalıdır.

Origin denetimi

WebSocket bağlantıları tarayıcının aynı kaynak politikasına (*same-origin policy*) tabi değildir ve CORS mekanizması burada koruma sağlamaz. Kütüphanelerin **CheckOrigin** benzeri denetimi varsayılan olarak her kaynağı kabul edecek biçimde bırakılırsa, başka bir sitedeki sayfa kullanıcının çerezleriyle bağlantı açabilir. Bu saldırıya siteler arası WebSocket ele geçirme (*cross-site WebSocket hijacking*) denir. Denetim, izin verilen kaynakların açık listesiyle yapılır.

İlişkisel veritabanı: dinamik tanımlayıcılar

Parametrelili sorgu kullanıldığı sürece klasik enjeksiyon riski düşüktür; sürücü, değeri sorgu metninden ayrı gönderir. Açık hemen her zaman parametre olarak bağlanamayan yerlerde oluşur: sıralama sütunu, filtrelenecek alan adı, tablo ya da şema adı. Bunlar sorgunun yapısına ait olduğu için bağlanamaz ve kod dizge birleştirmeye düşer.

Kural şudur: dinamik tanımlayıcılar kullanıcı girdisinden geçmez, izin listesindeki sabit değerlere eşlenir. İstemciden gelen `sort=created_at` değeri sorguya yazılmaz; önceden tanımlı bir eşleme tablosunda karşılığı aranır, bulunamazsa istek reddedilir.

İstemci tarafı

Çerçevelerin varsayılan sanitizasyonunu açıkça devre dışı bırakan çağrılar — Angular’daki `bypassSecurityTrustHtml` gibi — adlarında zaten uyarı taşır. Bu çağrılar korumayı kapattığı için, kod tabanında geçtikleri her yer ayrı ayrı gerekçelendirilmelidir.

Ayrı bir nokta, arayüzdeki rol bazlı gizlemenin bir güvenlik kontrolü olmadığıdır. Bir düğmenin gizlenmesi görünüm mantığıdır; uç noktayı korumaz, yalnızca kullanıcının onu tesadüfen tetiklemesini engeller. İstemci tarafındaki durum yönetiminin ne kadar tutarlı olduğu bunu değiştirmez, çünkü istekler arayüz üzerinden gelmek zorunda değildir.

Kayıt, izleme ve denetim izi

Yetersiz kayıt ve izleme, 2021 listesinde ayrı bir kategoridir ve veriyle test edilmesi güç olduğu için topluluk anketiyle listeye alınmıştır (OWASP, 2021). Güvenlik dışı bir gerekçeyle de savunulabilir: bir varlığın kim tarafından, ne zaman, hangi durumdan hangi duruma geçirildiğinin kaydı, olay sonrası geriye dönük izlenebilirlik için gereklidir. Güvenlik alanının denetim izi (*audit trail*) disiplini bu ihtiyaca doğrudan karşılık gelir.

Aynı bölümün ters yönü de geçerlidir: belirteçler, ön imzalı bağlantılar ve kimlik bilgileri kayıt dosyalarına yazılmaz. Kayıt tutmanın kendisi, kayıt tutulan yerin gizli veri deposuna dönüşmesine yol açmamalıdır.

Kaynak tüketimi

Medya işleyen sistemlerde girdi doğrulaması yalnızca biçime değil boyuta ilişkindir. Yükleme boyutu üst sınırı, eşzamanlı iş sayısı sınırı ve dönüştürme işlemleri için zaman aşımı tanımlanmadığında, tek bir dosya bütün işçi süreçleri kilitleyebilir. Bu durum kötü niyet gerektirmez; birinin yanlışlıkla çok büyük bir dosya yüklemesi yeterlidir. Kaynak tüketimi API Security Top 10’da ayrı bir madde olarak yer alır ve kullanılabilirliği doğrudan etkilediği için üretim sistemlerinde erken ele alınır.

Sonuç

Top 10'un üretim yöntemini bilmek, listenin içeriğini ezberlemekten daha fazla iş görür. Yöntem bilindiğinde listenin nerede güçlü, nerede kör olduğu da bilinir: insidans oranı ölçüsü, araç verisinin insan verisini bastırmasını engelleyerek sıralamayı riske yaklaştırır; ancak veri kümesi otomatik olarak test edilebilen şeylerle sınırlı olduğu için, test yöntemi henüz olgunlaşmamış zayıflıklar listede görünmez. Topluluk anketiyle eklenen iki kategori bu boşluğu kısmen kapatır, fakat aynı hamle listeyi saf ölçüm olmaktan çıkarır. Belgenin denetim ölçütü olarak kullanılamamasının nedeni budur ve bu bir eksiklik değil, tasarım kararının doğal sonucudur.

Bundan çıkan pratik ayrım şudur: Top 10 nereye bakılacağını, ASVS neyin doğrulanacağını, Cheat Sheet Series nasıl yapılacağını söyler. Bir kuruluşun bu üçünü aynı işlev için kullanmaya çalışması, her üçünden de az verim alması sonucunu doğurur.

Somut bulgular düzeyinde, güvenlik alanının ürettiği önerilerin belirgin bir kısmı tasarım disiplinlerinin bağımsız olarak vardığı sonuçlarla örtüşür. Nesne düzeyinde yetkilendirmenin imzaya taşınması, girdi tipinin alan tipinden ayrılması ve tanımlayıcıların kullanıcı girdisinden türetilmemesi — üçü de aynı ilkenin örnekleridir: bir hata sınıfı, örnekleri denetlenerek değil, sınıfın kendisi imkânsızlaştırılarak kapatılır. Denetim eklemek, denetimi unutma olasılığını da ekler; tipin ya da imzanın zorunlu kıldığı şey unutulamaz.

Bu çerçevenin sınırı, imza ve tip düzeyinde ifade edilemeyen kontrollerdedir. Süre sınırları, kaynak kotaları ve kayıt disiplini bu türdendir: bunlar derleyicinin zorlayamayacağı, gözden geçirme ve işletim pratiğine bağlı kalan kontrollerdir. Aynı biçimde, uzun ömürlü bağlantılarda yetkinin tazelenmesi bir tip sorunu değil, yaşam döngüsü sorunudur.

Açık kalan sorulardan biri, listelerin çoğalmasının kendi başına bir sorun üretip üretmediğidir. Ana liste dışında API, mobil, LLM, CI/CD, konteyner, sunucusuz, istemci tarafı ve daha fazlası için ayrı “Top 10” listeleri bulunmaktadır. Her biri kendi alanında yerinde olsa da, toplamda hangi belgenin ne zaman açılacağı sorusu, belgelerin kendisinden bağımsız bir yönlendirme ihtiyacı doğurur; Application Security Wayfinder ve Developer Guide gibi üst belgelerin varlığı bu ihtiyacın kabul edildiğini gösterir.

Kaynaklar

- OWASP (2021). *OWASP Top 10:2021 — Introduction*. OWASP Foundation. owasp.org/Top10/2021/A00.2021.Introduction/
- OWASP (2025). *OWASP Top 10:2025*. OWASP Foundation. owasp.org/Top10/2025/
- OWASP. *Application Security Verification Standard (ASVS)*. OWASP Foundation.
- OWASP. *API Security Top 10*. OWASP Foundation.
- OWASP. *Cheat Sheet Series*. OWASP Foundation.

- OWASP. *Web Security Testing Guide (WSTG)*. OWASP Foundation.
- OWASP. *Mobile Application Security (MAS)*. OWASP Foundation.
- OWASP. *Software Assurance Maturity Model (SAMM)*. OWASP Foundation.
- OWASP. *CycloneDX Bill of Materials Standard*. OWASP Foundation.
- MITRE. *Common Weakness Enumeration (CWE)*. cwe.mitre.org